Measuring a Synchronous API Contract

Laboratory 1
CMPS4191: Advanced Web Technologies
University of Belize

Andres Hung & Jennessa Sierra

https://github.com/andreshungbz/cmps4191-lab-01-synchronous-api

Aug 19, 2026

Table of Contents

Experimental Design

Research Question

How does report generation time affect the externally observable behavior of the synchronous report endpoint?

Hypothesis

If report generation time increases, then the time for the report endpoint to return a report also increases because synchronous endpoints require the client to wait for the server to finish processing before receiving a response.

Variables

Role	Variable
Independent	Artificial report-generation delay: 0s, 3s, 7s, 12s
Dependent	Response time, HTTP status, response bytes, complete report received, server completion log
Controlled	Endpoint, request body, consumer, reporting period, database contents, client command, server timeout

Table 1: List of independent, dependent, and controlled variables.

Predictions

Delay	Predicted Response Time	Predicted HTTP Status	Complete Body?
0s	0.1s	200 OK	Yes
3s	3.1s	200 OK	Yes
7s	7.1s	200 OK	Yes
12s	Fail	500 Internal Server Error	No

Table 2: Predictions of the response time, HTTP status, and complete response body for each API server run varying by delay.

The current design is predicted to fail after 10 seconds, since that is set as the server's write timeout and about the limit for holding a user's attention, according to Nielsen (2010). The design becomes unreliable because, after that point, the user may mistakenly believe that report generation has failed. Afterward, they are more likely to close the page (and thus the processing request) or send another request to generate the report again, both of which waste processing power. The user may even give up on the request, believing the system to be malfunctioning.

Procedure

1. The API server connected to the *cmps4191_lab_01* database was run four times, with the *report-delay* flag set to increasing values. The values used were 0, 3, 7, and 12 seconds.
2. For each API server run, the *curl* command was executed 3 times, using the *request.json* file as input to the */v1/reports* endpoint.
3. After each request, the time result and server log were recorded, and the *cat* command was used to inspect the response body.
4. Between each API server run, the server was stopped and restarted.

Results

Raw Measurements

Condition: 0 seconds

Run	HTTP Status	Time (s)	Bytes	Complete Report?	Server Logged Finish?
1	200	0.017454	366	Yes	Yes
2	200	0.005299	366	Yes	Yes
3	200	0.001810	366	Yes	Yes

Table 3: Raw measurement results for the API server run with 0s delay report generation.

Condition: 3 seconds

Run	HTTP Status	Time (s)	Bytes	Complete Report?	Server Logged Finish?
1	200	3.009481	366	Yes	Yes
2	200	3.003827	366	Yes	Yes
3	200	3.003708	365	Yes	Yes

Table 4: Raw measurement results for the API server run with 3s delay report generation.

Condition: 7 seconds

Run	HTTP Status	Time (s)	Bytes	Complete Report?	Server Logged Finish?
1	200	7.009285	366	Yes	Yes
2	200	7.004174	366	Yes	Yes
3	200	7.004008	366	Yes	Yes

Table 5: Raw measurement results for the API server run with 7s delay report generation.

Condition: 12 seconds

Run	HTTP Status	Time (s)	Bytes	Complete Report?	Server Logged Finish?
1	000	12.003583	0	No	Yes
2	000	12.002592	0	No	Yes
3	000	12.002204	0	No	Yes

Table 6: Raw measurement results for the API server run with 12s delay report generation.

Mean Response Times

Delay	Mean Response Time	Successful Responses	Complete Reports
0s	0.008188	3 / 3	3 / 3
3s	3.005672	3 / 3	3 / 3
7s	7.005822	3 / 3	3 / 3
12s	12.002793	0 / 3	0 / 3

Table 7: Calculated Mean Response Time results for each API server run varying by delay. Note: Mean Response Time was calculated by the formula: $(T_1 + T_2 + T_3) / 3$, where T is a recorded time. Results were rounded to the 6th decimal point.

Observations

The API response time was near-instantaneous, with an initial report delay of 0s. When the report delay increased from 0s to 3s, the mean response time for report generation changed from approximately 0.008188s to 3.005672s. This increasing trend is also seen for the other report delay increases. From an increase in report delay from 3s to 7s, the mean response time increased from approximately 3.005672s to 7.005822s. All requests under these conditions returned successful and complete reports. At the 12s delay conditions, however, the client did not receive a successful complete response due to the 10s write timeout configuration.

Explanation

The evidence shows that response time increases approximately with report-generation time, from a mean of 0.008188s at 0s delay to 12.002793s at 12s delay. This exposes a coupling between the lifetime of the HTTP request and the lifetime of report generation, since the client must remain waiting until the report is complete. Therefore, the system requires that report requests be acknowledged regardless of how long it takes to generate the report. The API should consequently be revised so that accepted work is acknowledged before the completed report is available, using applicable mechanisms such as a background worker or job system. From the observations, explanations can be made for the following mechanisms:

1. Why did the response time change as the artificial delay increased?

The response time changed (increased) as the artificial delay increased because the Synchronous API communication style was used in the API server implementation. In the HTTP handler used for report generation, processing starts and completes before the HTTP response result is written back to the client who sent the HTTP request. If the processing time increases, as simulated by the artificial delay increase, then the client must wait that time, thus changing (increasing) the response time.

2. What must the handler complete before it can send 200 OK with the report?

The handler must complete report generation before sending a 200 OK with the report. The report generation process involves executing a query against a database, instantiating an object (struct) into which the report values are scanned, and checking for errors.

3. While the report is being generated, is the original request complete or outstanding?

It is outstanding. The API server does not send an HTTP response until report generation is complete. This means that if report generation exceeds the time the server can successfully write the response, the client may not receive the completed report, even though server-side processing may continue.

4. Would asynchronous execution make the report faster, or would it change when the client receives an acknowledgment?

Asynchronous execution would not make report generation faster, as that depends on how optimized the database query is, server processing power, etc. However, it would make the client receive an acknowledgment significantly earlier. This earlier acknowledgment helps keep the API responsive and avoid potential timeouts, as observed.

Limitations

This experiment includes some limitations, listed below:

- The experiment was conducted with services entirely local to a single machine, namely the API server application, the database, and tests with the curl command. In a production environment, where the server, database, and clients may be in three different locations, the real response times would vary more due to the network traversal required.
- Actual report-generation time can be affected by extraneous system and environmental variables, such as server load, database performance, query complexity, and network latency. These factors were not independently varied or measured in this experiment, so the artificial delays may not fully represent response times in a real production environment.

Derived Requirements

- A valid report request should be acknowledged <u>within 1 second, independent of the report generation duration</u>.
- Report generation should not depend on <u>the client waiting for the entire duration of the report generation process</u>.
- The client should be able to determine <u>the status of report generation, such as whether it is still in progress or if it failed.</u>
- The system should preserve <u>report generation requests and their outcomes</u>.
- Report generation duration should not determine <u>how long it takes for the client to receive an acknowledgment</u>.

Revised Contract Proposal

- [ ] **Current Contract**: Return 200 OK with the completed report.
- [x] **Revised Contract**: Acknowledge accepted work before the report is complete.

The current contract is being revised because long report generation times, as evidenced by observations and experimental results, are an issue affecting the client's perceived responsiveness. Additionally, if a report takes too long to generate, timeouts can occur even if the generation is ultimately successful. The revised contract would allow the feedback given to the client to be independent of the processing required for report generation. This decoupling would also increase the flexibility and scalability of contract and requirement implementation.

References

Nielsen, J. (2010, June 20). *Website response times*. Nielsen Norman Group. https://www.nngroup.com/articles/website-response-times/